

Cryptographic Sovereignty, Architectural Vulnerabilities, and the Regulatory Imperative: A Multi-Dimensional Investigation into Secure Messaging Frameworks and Indian Cyber Law

**An Academic Research Report on Multi-Layer Data Pipelines,
Regulatory Compliance, and Decentralized Networks**

Submitted by: Ankit Kumar

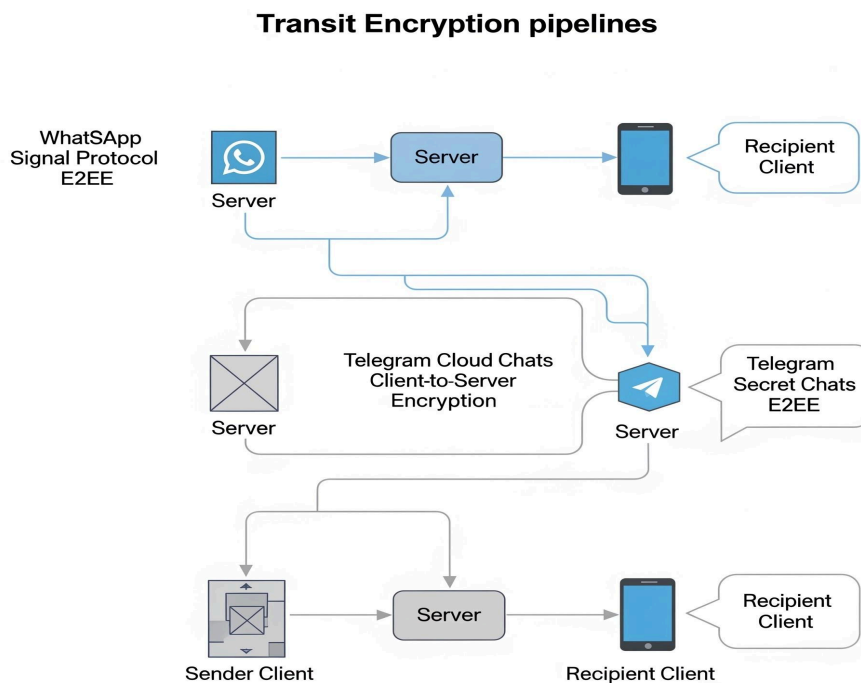
Author & Principal Systems Architect

B.Sc. (Hons.) Computer Science and Data Analytics
Indian Institute of Technology Patna (IIT Patna)

Date: 23 June 2026

Comprehensive Architectural Comparison

The modern instant messaging landscape is a complex convergence of distributed systems engineering, cryptographic protocol design, and persistent storage paradigms. To evaluate the security and privacy profiles of these platforms, it is necessary to examine the architectural pipelines, transit encryption mechanics, and database schemas of Meta's WhatsApp, Instagram Direct Messages (DMs), and Telegram FZ LLC's Telegram (differentiating between Cloud Chats and Secret Chats).



Transit Encryption and Cryptographic Primitives

The mathematical mechanisms securing data in transit vary significantly across these platforms. WhatsApp enforces end-to-end encryption (E2EE) by default using the Signal Protocol¹. Telegram uses a hybrid model, defaulting to client-to-server encryption for "Cloud Chats" and reserving E2EE for manually initiated "Secret Chats"³. Instagram DMs have transitioned to transport-only TLS encryption following a major structural shift in 2026⁵.

The Signal Protocol: WhatsApp's Implementation

WhatsApp's default E2EE model relies on the Signal Protocol to guarantee confidentiality, perfect forward secrecy (PFS), and post-compromise security (PCS)². The protocol achieves

this through a multi-stage cryptographic handshake and a continuous key-rotation mechanism².

Extended Triple Diffie-Hellman (X3DH) Agreement

To establish an asynchronous cryptographic session key when one party is offline, the protocol utilizes the X3DH key agreement protocol². Each user registers a set of elliptic curve keys (using Curve25519) with WhatsApp's central identity directory²:

- **Identity Key (IK)**: A long-term static Curve25519 key pair unique to the user device².
- **Signed Pre-Key (SPK)**: A medium-term Curve25519 key pair signed by the static Identity Key to prevent identity spoofing².
- **One-Time Pre-Keys (OPK)**: A pool of ephemeral Curve25519 key pairs generated by the client, used once and deleted upon consumption to preserve cryptographic entropy².

When Alice initiates a session with Bob, she requests Bob's active pre-key bundle from the central server². She generates her own ephemeral Curve25519 key pair (EK_A) and computes a master shared secret (SS) via a series of concatenated Diffie-Hellman (DH) operations³:

$$DH_1 = \text{ECDH}(IK_A, SPK_B)$$

$$DH_2 = \text{ECDH}(EK_A, IK_B)$$

$$DH_3 = \text{ECDH}(EK_A, SPK_B)$$

$$DH_4 = \text{ECDH}(EK_A, OPK_B) \quad (\text{if } OPK_B \text{ is present in the bundle})$$

The master shared secret (SS) is then derived using a Key Derivation Function (KDF), typically HKDF-SHA256²:

$$SS = \text{HKDF}(DH_1 \parallel DH_2 \parallel DH_3 \parallel DH_4, \text{salt}, \text{info})$$

The Double Ratchet Algorithm

Once the initial session is established, the Double Ratchet updates session keys on a per-message basis². This combines a Symmetric-Key Ratchet (based on KDF chains) with an active Diffie-Hellman Ratchet².

The Symmetric-Key Ratchet is a one-way KDF chain². For each sent or received message, a new message key (MK) is derived, and the chain key (CK) is advanced²:

$$MK = \text{HMAC-SHA256}(CK_i, \text{"message-key-salt"})$$

$$CK_{i+1} = \text{HMAC-SHA256}(CK_i, \text{"chain-key-salt"})$$

When Alice receives a message from Bob, Bob attaches a new Diffie-Hellman public key (DH_{pub_Bob}) to the payload³. The client executes a DH ratchet step, deriving a new root key and a new chain key, which resets the symmetric KDF chain². This dual-ratchet architecture ensures that even if a message key is compromised, prior and future messages remain secure². Payloads are encrypted using AES-256-GCM, with the derived message key and a unique, sequence-based initialization vector (IV)².

Telegram and MTProto 2.0

Telegram utilizes a proprietary cryptographic suite named MTProto 2.0⁴.

Standard Cloud Chats (Client-to-Server)

By default, Telegram chats do not use end-to-end encryption³. Instead, they rely on transport-layer client-to-server encryption⁴. During registration, the client and server negotiate a 2048-bit static Authorization Key ($AuthKey$) using a standard Diffie-Hellman key exchange³.

For each payload, the client calculates a 128-bit Message Key ($MsgKey$) as the middle 128 bits of a SHA-256 hash of the message body, padding bytes, and a slice of the pre-negotiated $AuthKey$ ⁷.

$$MsgKey = \text{SHA256}(AuthKey_{\text{slice}} \parallel \text{Payload} \parallel \text{Padding})_{64..191}$$

This $MsgKey$ is then used to derive the AES key and IV, which encrypts the payload using

AES-256 in Infinite Garble Extension (IGE) mode or AES-256-CTR⁷. Because the *AuthKey* is shared with the Telegram server infrastructure, the server can decrypt, index, and cache message payloads⁴. This design enables server-side features like global search, bot interactions, and cloud synchronization, but requires users to trust the platform's cloud security model⁴.

Secret Chats (E2EE)

When a user manually initializes a "Secret Chat," Telegram executes a client-to-client Diffie-Hellman key exchange over the transport layer³. The resulting *AuthKey* is stored exclusively on the two participating client devices and is not cached on Telegram's cloud servers³.

While this model achieves E2EE, its lack of a continuous per-message key-ratcheting mechanism (like the Double Ratchet) means that if a static session key is compromised, all messages within that session are exposed².

Meta's 2026 Structural Shift: The Decommissioning of E2EE on Instagram DMs

In May 2026, Meta removed optional End-to-End Encryption (E2EE) for Instagram Direct Messages (DMs)⁵. This decision reversed Meta's previous strategy of consolidating E2EE across its entire messaging portfolio⁵.

The primary drivers behind this architectural shift include:

- **Low Feature Adoption & Infrastructure Complexity:** Meta's internal telemetry indicated that only a small fraction of Instagram's user base actively opted into E2EE chats⁵. Maintaining separate E2EE and non-E2EE codebases, along with managing associated client-side cryptographic key stores, created significant infrastructure overhead and degraded overall application performance⁵.
- **The Take It Down Act (TIDA) Compliance Directive:** The primary regulatory driver was the compliance deadline of the United States' federal Take It Down Act (TIDA), which went into effect on May 19, 2026⁵. Under Section 3 of TIDA, covered platforms are required to establish an accessible, prompt notice-and-removal process to delete Non-Consensual Intimate Imagery (NCII)—including AI-generated deepfakes (termed "digital forgeries")—within 48 hours of notification¹². The statute mandates that platforms make "reasonable efforts" to identify and permanently eliminate all identical duplicate copies of reported media¹².
- **The Cryptographic Impasse of Content Moderation:** Under a client-side E2EE architecture, server-side content matching, media hash-database queries (e.g., matching against PhotoDNA or StopNCII/NCMEC hash registries), and duplicate-checking are

impossible because the platform does not have access to the unencrypted media blobs⁶. De-fanging E2EE on Instagram DMs allowed Meta to transition all traffic back to transport-only encryption⁵. This change gave Meta's server-side automated content moderation pipelines direct visibility into transit media, enabling them to compute cryptographic hashes of images and videos and cross-reference them against NCII databases within the statutory 48-hour compliance window⁵. Users requiring E2EE were directed to transition their conversations to WhatsApp⁵.

Data-at-Rest and Storage Architecture

The structural differences between WhatsApp and Telegram extend to their persistent storage models, local database files, and cloud-scaling designs.

WhatsApp: Stateless Server and Local SQLCipher Model

WhatsApp utilizes a local-first, decentralized storage model. Messages in transit are stored temporarily in queue buffers on WhatsApp's servers and are deleted immediately upon receiving delivery confirmation from the recipient client².

Local Database Structure

On Android, data-at-rest is stored within an encrypted SQLite database file located at `/data/data/com.whatsapp/databases/msgstore.db` (with backups written as `.crypt14` or `.crypt15` arrays)¹⁷. On iOS, the data resides within the AppGroup sandbox container as a CoreData SQLite store at `AppDomainGroup-group.net.whatsapp.WhatsApp.shared/ChatStorage.sqlite`¹⁷.

SQLCipher Security Layer

The database is encrypted using SQLCipher 4²⁰. By default, SQLCipher uses AES-256-CBC with key derivation handled via PBKDF2 with HMAC-SHA512 over 256,000 iterations and a 4096-byte database page size²¹.

Hardware-Backed Cryptographic Key Protection

On Android, the raw 256-bit AES database key is stored within `/data/data/com.whatsapp/files/key`¹⁸. To secure this key, WhatsApp utilizes the Android Keystore System¹⁸. This system stores the master key within a hardware-isolated environment, such as a Trusted Execution Environment (TEE) or StrongBox Keymaster²³. On iOS, the key is secured using the Secure Enclave Processor (SEP) via the iOS Keychain Services wrapper API²³.

Telegram: Centralized Sharded Cloud Databases

Telegram employs a centralized, horizontally scalable cloud storage architecture to maintain user message histories, group states, and public channels⁹.

No-Encryption Default for Local Cache

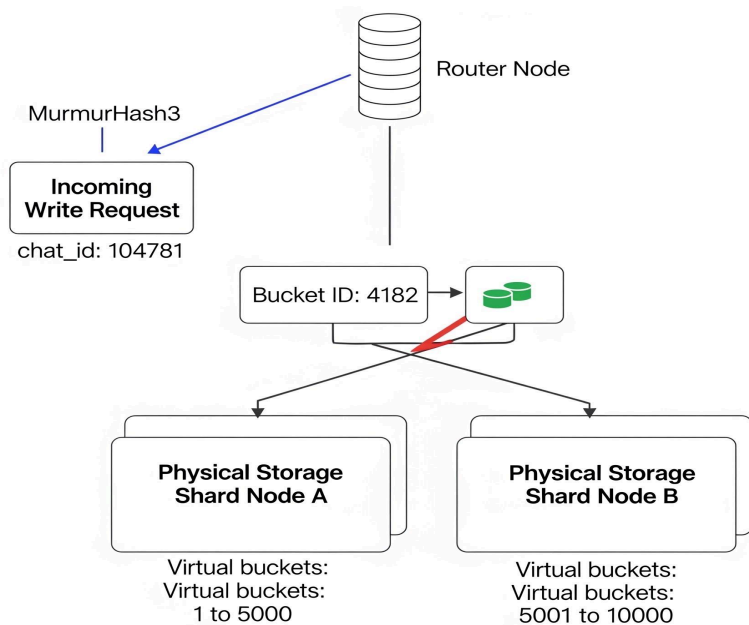
Telegram's standard client applications do not encrypt their local databases or message histories on the device²³. On Android, cached database elements, message snippets, and media catalogs are stored in plaintext or basic serialized state formats in cache.db at /data/data/org.telegram.messenger/files/cache.db¹⁷. Secret Chats are held in memory or local cache files but lack robust hardware-backed database encryption²³.

Server-Side Sharded Topology

Telegram relies on a distributed cluster of custom database engines to manage petabytes of real-time data across global datacenters⁹. A primary building block of this architecture is **Tarantool**, an in-memory database and application server optimized for horizontal scaling via sharding and master-master replication²⁴.

Telegram segments data logically into horizontal partitions (shards) across physical storage nodes using a hashed sharding strategy⁹. Utilizing Tarantool's vshard module, the database cluster abstracts physical nodes into thousands of virtual buckets²⁴. Data is balanced across these buckets using a sharding key, typically the user's registered ID (user_id or chat_id)⁹.

Tarantool Database Sharding Model



This guarantees that a user's entire non-E2EE message profile, contacts list, and channel history map to the same replica sets⁹. This approach enables low-latency read and write operations, as query routing proxies can resolve a query directly to a single physical node without performing expensive multi-node joins⁹.

Multi-Edit States and Database Consistency

Message modifications (edits) highlight the distinct database designs of WhatsApp and Telegram.

WhatsApp Distributed Edit Model

When a user edits a message on WhatsApp, the client generates a delta command packet containing the original message ID (`message_row_id` or `stanza_id`), a modified payload string, and an incremental edit sequence version counter. This packet is encrypted via the Double Ratchet and transmitted to the recipient².

Upon receipt, the recipient's local database engine executes an in-place modification. The messages table in `msgstore.db` is updated, and the original message body is overwritten or archived within a supplementary `message_add_on` or `message_quoted` table²². Because WhatsApp's servers are stateless, they do not retain a history of the edit states².

Telegram Centralized Multi-Edit Storage

In Telegram's cloud architecture, message editing triggers a state update on the sharded server database⁹. When an edit request is received, the server updates the Messages table for that specific `message_id` on the target Tarantool shard⁹.

Crucially, the server maintains consistency across its sharded cluster by recording edit states and histories in internal transaction logs²⁴. This ensures that when a client requests updates via synchronization pings, the database router returns the latest message state. However, this centralized state management can introduce replication delays or synchronization mismatches if the platform's real-time features are modified under regulatory duress, as seen during the June 2026 NEET-UG litigation in India²⁷.

Architectural Specifications: Core Message and Contact Tables

WhatsApp Forensic Schema (Local SQLite - Android)

The following schema represents the core message storage and contact synchronization models extracted from `msgstore.db` and `wa.db`¹⁷.

SQL

```
-- Located in: msgstore.db
CREATE TABLE messages (
  _id INTEGER PRIMARY KEY AUTOINCREMENT,
  chat_row_id INTEGER NOT NULL,
  key_remote_jid TEXT NOT NULL,
```



```

    key_from_me INTEGER NOT NULL, -- 1 = Sent, 0 = Received
    key_id TEXT NOT NULL, -- Unique Stanza ID
    status INTEGER, -- 0=Received, 4=Server-Pending, 5=Delivered, 13=Read
    needs_push INTEGER,
    data TEXT, -- Plaintext body of the message (locally decrypted)
    timestamp INTEGER NOT NULL, -- Unix timestamp in milliseconds
    media_wa_type TEXT, -- '0'=Text, '1'=Image, '2'=Audio, '3'=Video
    media_url TEXT,
    media_mime_type TEXT,
    media_size INTEGER,
    message_add_on_flag INTEGER DEFAULT 0,
    FOREIGN KEY(chat_row_id) REFERENCES chat(_id)
);

```

```

CREATE TABLE chat (
    _id INTEGER PRIMARY KEY AUTOINCREMENT,
    jid TEXT UNIQUE NOT NULL, -- Destination JID (e.g.,
[phone_number]@s.whatsapp.net)
    subject TEXT, -- Group name if applicable
    last_message_row_id INTEGER,
    unseen_message_count INTEGER
);

```

```

CREATE TABLE message_add_on (
    _id INTEGER PRIMARY KEY AUTOINCREMENT,
    message_row_id INTEGER NOT NULL,
    add_on_type INTEGER, -- 1=Reaction, 2=Disappearing Keep Flag
    sender_jid_row_id INTEGER,
    timestamp INTEGER,
    text_data TEXT, -- Holds raw reaction emoji or metadata
    FOREIGN KEY(message_row_id) REFERENCES messages(_id) ON DELETE
CASCADE
);

```

```

-- Located in: wa.db
CREATE TABLE wa_contacts (
    _id INTEGER PRIMARY KEY AUTOINCREMENT,
    jid TEXT UNIQUE NOT NULL, -- Contact JID
    phone_number TEXT,
    display_name TEXT,
    status TEXT, -- User "About" status
    status_timestamp INTEGER
);

```

Telegram Distributed Schema (Logical Representation of Sharded Tarantool Model)

The following schema represents the server-side transactional message storage and distributed routing engine managed across sharded datacenters⁹.

SQL

```
-- Distributed across logical vshard buckets
CREATE TABLE Users (
  user_id INT8 PRIMARY KEY, -- 64-bit integer
  bucket_id INT4 NOT NULL, -- Tarantool vshard routing key
  username VARCHAR(32),
  phone_number VARCHAR(16) UNIQUE,
  first_name VARCHAR(64),
  last_name VARCHAR(64),
  profile_picture_url VARCHAR(256),
  status_state INT2, -- 1=Online, 2=Offline, 3=Recently
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP
) WITH (sharding_key = 'user_id');

CREATE TABLE Messages (
  message_id INT8 PRIMARY KEY,
  bucket_id INT4 NOT NULL,
  chat_id INT8 NOT NULL, -- Maps to a User, Group, or Channel ID
  sender_id INT8 NOT NULL,
  content TEXT,
  media_url VARCHAR(512),
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  edit_timestamp TIMESTAMP NULL, -- Populated and updated when
message is edited
  edit_count INT2 DEFAULT 0, -- Tracks number of times message was
modified
  status_bitmask INT4 NOT NULL -- Flags: 1=Sent, 2=Delivered,
4=Read, 8=Edited
) WITH (sharding_key = 'chat_id');

CREATE TABLE Group_Participants (
  group_id INT8 NOT NULL,
  user_id INT8 NOT NULL,
  role_flags INT4 NOT NULL, -- Admin privileges, custom titles
```

```

joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (group_id, user_id)
) WITH (sharding_key = 'group_id');

```

Architectural Platform Matrix

Architectural Vector	WhatsApp (Signal Protocol)	Telegram Cloud Chats (MTPROTO 2.0)	Telegram Secret Chats (MTPROTO 2.0)	Instagram DMs (Post-May 2026 Shift)
Transit Encryption Type	End-to-End Encryption (E2EE) ¹	Client-to-Server Encryption ⁴	End-to-End Encryption (E2EE) ³	Transport-Only (Standard TLS) ⁵
Cryptographic Primitives	Curve25519, HKDF-SHA256, AES-256-GCM, HMAC-SHA256 ²	SHA-256, AES-256-IGE/CTR ⁷	DH-2048, SHA-256, AES-256-IGE/CTR ³	RSA-4096 / ECDSA-384, AES-128/256-GCM ⁵
Key Ratcheting / PFS	Double Ratchet (Per-message rotation with PFS/PCS) ²	None (Static authorization session keys) ⁴	Handshake-bound DH Session Key ³	Handshake-bound TLS Session Key ⁵
Server-Side Payload State	Stateless (Zero payload storage post-delivery) ¹⁶	State-Persistent (Messages stored long-term on servers) ⁹	Stateless (Zero payload storage on servers) ⁴	State-Persistent (Messages stored on Meta servers) ⁵
Database-at-Rest Protection	Locally Encrypted via SQLCipher 4 ²⁰	Unencrypted (Plaintext local SQLite cache) ²³	Unencrypted (Plaintext local SQLite cache) ²³	Sandbox-enforced unencrypted SQLite stores ⁶
Hardware Key	Android	None ²³	None ²³	iOS Keychain SEP /

Integration	Keystore TEE / iOS Secure Enclave SEP ²³			Android Keystore TEE ⁶
Server Database Topology	Stateless relay queues (No persistent server DB) ²	Horizontal Sharding via Tarantool vshard clusters ⁹	Stateless relay queues (No persistent server DB) ⁴	Centralized, distributed relational database systems ⁵

Traceability, Metadata, and the Ad-Targeting Illusion

The security guarantees of End-to-End Encryption (E2EE) are often conflated with comprehensive privacy protection. While E2EE successfully hides the *contents* of a message from intermediaries, it does not obscure the transactional metadata generated by network interactions². In the context of metadata analysis and tracking, this distinction is critical.

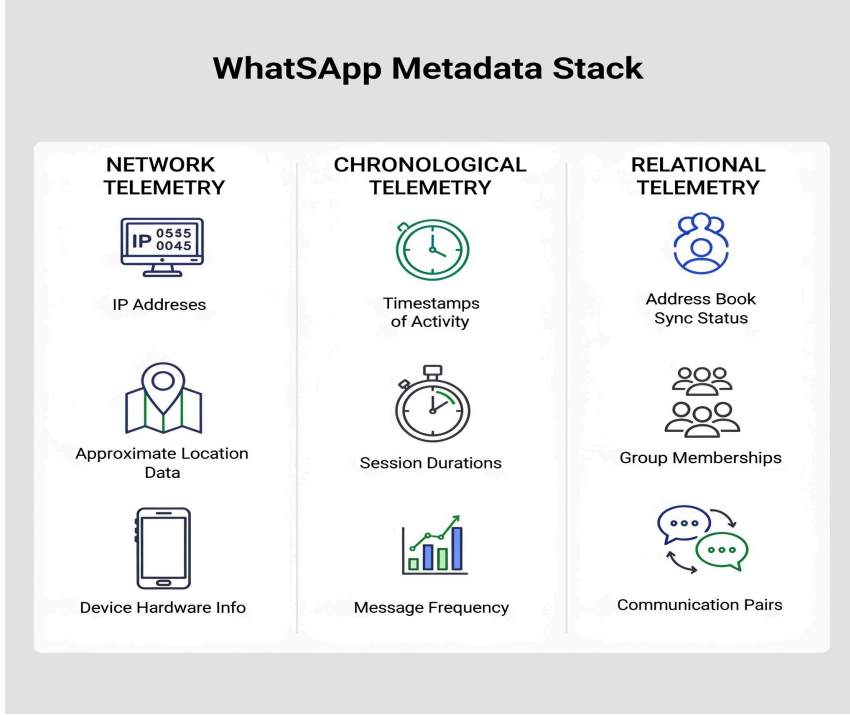
Zero-Knowledge Message Bodies vs. Operational Metadata Exfiltration

WhatsApp operates a zero-knowledge transit network regarding message contents¹. However, it collects, parses, and retains a detailed log of non-encrypted operational metadata².

When a law enforcement agency or government entity presents a valid legal order to WhatsApp's Law Enforcement Response Team (LERT), WhatsApp cannot produce message bodies¹. It can, however, provide transaction logs containing the following metadata elements¹:

- **Subscriber Information:** Registered phone number, name, email addresses, platform creation timestamp, last-seen online timestamp, and device-specific hardware profiles (e.g., OS version, device manufacturer, device model, and push tokens)¹.
- **IP Connection History:** Real-time IP addresses recorded during socket connections and voice/video sessions¹.
- **Social Graph Topology:** The target user's complete contact directory (both WhatsApp and non-WhatsApp phone numbers) and a log of all group chats the user belongs to, including group creator identifiers, admin rosters, participant lists, and creation timestamps¹.
- **Traffic Logs (Transactional Metadata):** A sequential timeline recording the time of communications, call durations, and message delivery flows. This log records who contacted whom, when, and how often².

WhatsApp Metadata Stack



Mathematical Models of Metadata Attack

The utility of metadata for surveillance and behavioral profiling can be demonstrated through two analytical methods.

Graph-Theoretic Social Networks: Betweenness Centrality

A surveillance target's structural influence and communication flow within a decrypted social graph can be identified without analyzing message contents². Let a communication network be

represented as a graph $G = (V, E)$, where V is the set of vertices (users) and E is the set of edges (directed/undirected message transmissions)². The **Betweenness Centrality**

$C_B(v)$ of a node $v \in V$ is calculated as²:

$$C_B(v) = \sum_{s \neq v \neq t \in V} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

Where σ_{st} is the total number of shortest paths from node s to node t , and $\sigma_{st}(v)$ is the number of those paths that pass through v ². By mapping these transactional flows, law

enforcement can identify key coordination hubs within criminal or activist networks with high precision, even when all payloads are fully encrypted².

Bayesian Temporal Correlation: Target Inference

Bayesian inference can correlate non-encrypted metadata points to determine highly sensitive attributes (e.g., romantic partnerships, medical status, political affiliations) with high

probability². Let C be a specific behavioral pattern or state (e.g., a medical diagnosis), and let $M_1, M_2, \dots, M_n$ be observed metadata events (e.g., a midnight connection to a specialized helpline, proximity logs, or correlation of online activity between two devices)². The posterior probability $P(C | M_1 \cap M_2 \cap \dots \cap M_n)$ is evaluated using Bayes' Theorem²:

$$P(C | M_1 \cap \dots \cap M_n) = \frac{P(M_1 \cap \dots \cap M_n | C) \cdot P(C)}{P(M_1 \cap \dots \cap M_n)}$$

Assuming conditional independence of metadata events given the class C , this simplifies to²:

$$P(C | M_1 \cap \dots \cap M_n) = \frac{P(C) \prod_{i=1}^n P(M_i | C)}{P(M_1 \cap \dots \cap M_n)}$$

By chaining multiple independent metadata observations (such as matching connection times between a patient and a doctor), this model can infer sensitive target behaviors with high statistical confidence².

The "Ad-Targeting Illusion"

Users often report receiving targeted advertisements on Instagram immediately after discussing specific products within end-to-end encrypted WhatsApp chats. This phenomenon has fueled the belief that Meta actively decrypts and reads WhatsApp messages². However, this is an "Ad-Targeting Illusion" driven by cross-app telemetry, graph-theoretic user grouping, and behavioral tracking pipelines across Meta's ecosystem².

The underlying data integration pipelines include:

- **Deterministic Account Synchronization:** Meta links WhatsApp accounts to Facebook and Instagram profiles using verified matching identifiers, such as registered phone numbers, email addresses, and hardware serials (IMEIs)². This enables direct synchronization of telemetry across platforms².
- **Network Co-location Grouping:** Meta's apps collect spatial and network telemetry². By correlating shared external IP addresses, local subnet Wi-Fi SSIDs, and Bluetooth Low

Energy (BLE) beacon scans, Meta's backend algorithms can group devices together in the same physical space². If User A searches for a specific product on an external browser, Meta's system can target ads for that product to User B (a close contact who was recently co-located with User A)².

- **Meta Pixel and SDK Telemetry:** Over 30% of commercial websites embed the Meta Pixel (a tracking script) or the Meta Business SDK. When a user browses a product on an e-commerce platform, their browser transmits behavioral events (e.g., ViewContent, AddToCart) along with their hashed email or phone number to Meta's ad engine. This lookup matches the web action to the user's WhatsApp-associated phone number, triggering corresponding ads on Instagram without needing to access E2EE chats.
- **Meta AI & Contextual Interaction Logs:** Interactions with Meta AI inside WhatsApp (such as using @MetaAI in group chats or direct queries) bypass E2EE². Telemetry and query strings submitted to these generative assistants are processed in plaintext on Meta's server infrastructure and are integrated directly into the user's ad-targeting profile².

The Notification History Leak Endpoint Vulnerability

A critical cryptographic endpoint vulnerability in modern mobile operating systems (Android and iOS) involves the handling of message previews via system notifications³⁵.

1. **Incoming Payload Decryption:** An incoming E2EE message is received by WhatsApp's background daemon and decrypted within the application's isolated memory sandbox².
2. **Notification API Handoff:** To display a heads-up banner to the user, the decrypted plaintext string is passed to the OS-level notification manager API (such as NotificationManager on Android or UNUserNotificationCenter on iOS)³⁵.
3. **Vulnerability Exposure (The Sandbox Bypass):** Once the string is transferred to the operating system, it leaves the application's secure sandbox³⁶. On Android, the OS writes these notification payloads directly into the system database logs³⁸.
4. **Malicious Interception:** Any third-party application on the device that has been granted the NotificationListenerService permission can read, parse, and exfiltrate these notification payloads globally³⁵. This permission allows malicious apps or ransomware (e.g., DroidLock variants) to capture incoming plaintext messages and one-time passwords (OTPs) without needing to compromise the E2EE application's sandbox or database³⁷.

Key Metadata Fields Logged by WhatsApp LERT Pipelines

Metadata Category	Variable Name	Technical Format	Explicit Value Example	Privacy & Graph Implications
Subscriber Information	JID	String	919876543210@s.whatsapp.net	Unique global cryptographic routing identifier used to link accounts across Meta platforms ¹ .
Temporal Cadence	LastSeen	Epoch Milliseconds	1781200860000	Exposes precise behavioral patterns and active sleep schedules to correlation attacks ¹ .
Network Identification	External_IP	IPv4 / IPv6 String	203.0.113.84	Discloses the user's geographic location, ISP, and network co-location clusters ¹ .
Device Identification	IMEI	15-Digit Numeric	357924108642013	Uniquely identifies the physical hardware, facilitating device tracking across network interfaces ² .
Relational Topology	Group_Roster	Array of JIDs	[9198..., 9199...]	Maps the user's social graph and community networks ¹ .
Traffic Metadata	Stanza_Flow	Sequence State	ID: 8F4C - Sent	Details communication volumes and delivery flows without decrypting the payload ² .

Statutory Enforcement and Indian Cyber Law Frameworks

The intersection of secure messaging architectures and sovereign law enforcement is

governed primarily by the **Information Technology (IT) Act, 2000** of India and its delegated rules.

Section 69A: Scope, Executive Powers, and Constitutional Limits

Section 69A of the IT Act, 2000 empowers the Central Government to issue blocking directives to prevent public access to information through any computer resource⁴¹.

Statutory Grounds

The government can invoke Section 69A on six specific grounds⁴¹:

1. Sovereignty and integrity of India⁴¹.
2. Defense of India⁴³.
3. Security of the State⁴¹.
4. Friendly relations with foreign States⁴³.
5. Public order⁴¹.
6. Prevention of incitement to the commission of any cognizable offense relating to the above⁴¹.

Executive Mechanism

The execution of Section 69A is governed by the **Information Technology (Procedure and Safeguards for Blocking for Access of Information by Public) Rules, 2009**⁴¹. Under Rule 9, a designated officer from the Ministry of Electronics and Information Technology (MeitY) can issue emergency blocking directives to intermediaries without a prior hearing, provided the case presents an urgent need²⁷. These directives are subsequently reviewed by an inter-ministerial committee within 48 hours²⁷.

Constitutional Boundaries

The constitutional validity of Section 69A was upheld by the Supreme Court of India in *Shreya Singhal v. Union of India (2015)*⁴⁷. The Court ruled that Section 69A has sufficient procedural safeguards, including written justifications and mandatory review committee assessments, aligning it with the reasonable restrictions on free speech permitted under Article 19(2) of the Constitution⁴⁷.

However, platform-wide blocking orders can conflict with the Fundamental Right to Privacy established in the landmark nine-judge bench ruling *Justice K.S. Puttaswamy v. Union of India*

(2017), which declared privacy to be an integral part of Article 21⁴⁹. Any state interference with privacy must satisfy the three-fold test of legality, legitimate state aim, and strict proportionality²⁷.

The Traceability Mandate vs. End-to-End Encryption

A significant point of tension between Indian cyber law and cryptographic design is **Rule 4(2)** of the **Information Technology (Intermediary Guidelines and Digital Media Ethics Code) Rules, 2021**⁴⁸.

Rule 4(2) Requirements

Rule 4(2) mandates that a Significant Social Media Intermediary (SSMI) primarily engaged in messaging services must enable the identification of the **first originator** of specific information within the territory of India when ordered by a competent court or authority⁴⁸. This power is restricted to the investigation of offenses of a threshold gravity (e.g., threat to sovereignty, security of the state, public order, or offenses relating to rape or child sexual abuse)⁵¹.

The Cryptographic Impasse

E2EE applications cannot comply with Rule 4(2) without undermining their security design⁴⁸.

- **Loss of Plausible Deniability:** The Double Ratchet protocol ensures that once a message key is used, it is deleted, and messages cannot be mathematically linked back to a sender using key-reuse trails². To support traceability, the platform would have to sign each message with a persistent cryptographic identity key or attach a tracing token to all forwarded content⁵³.
- **The "Forwarding Hash" Storage Burden:** To trace the first originator of a viral forward, platforms would be forced to index and store cryptographic hashes of every message sent on their network in centralized databases, comparing them whenever content is forwarded⁵³. This mechanism breaks the zero-knowledge model and introduces a major target for state-sponsored or malicious hacking².

Case Study: *Telegram FZ LLC v. Union of India* (W.P.(C) 8259/2026)

In June 2026, the Delhi High Court issued a landmark ruling that redefined architectural liability for digital platforms under Indian law²⁷.

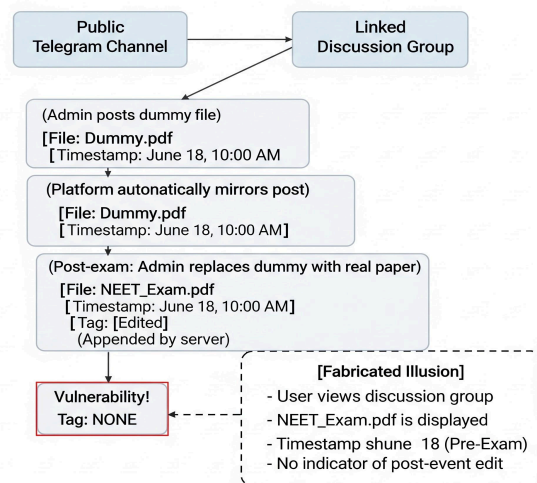
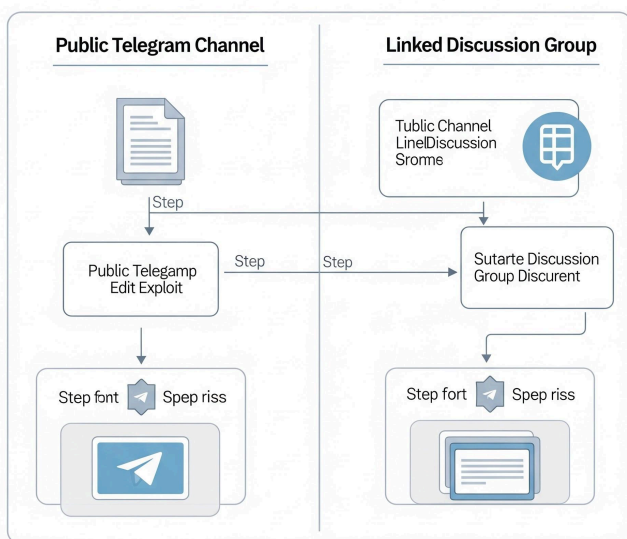
Factual Matrix and the NEET-UG Controversy

The National Testing Agency (NTA) cancelled the National Eligibility-cum-Entrance Test (Undergraduate) 2026 (NEET-UG 2026) scheduled for May 3 due to systemic paper leaks, scheduling a nationwide re-examination for June 21⁵⁴. In the days leading up to the re-test, security agencies identified organized networks selling leaked or fabricated exam materials on Telegram channels and groups²⁷.

The Timestamp Retention Edit Exploit

A key catalyst for the government's intervention was the systematic exploitation of Telegram's message-editing feature to generate fabricated evidence of exam paper leaks²⁸.

Telegram Timestamp Edit Exploit: Public Channel & Linked Discussion Group File Replacement



The exploit works as follows:

1. **Channel-Group Synchronization:** A scammer creates a public Telegram channel and links it to an associated discussion group, which mirrors the channel's posts³⁰.
2. **Pre-Event Dummy Upload:** Days before the exam (e.g., June 18 at 10:00 AM), the administrator uploads an innocuous dummy PDF to the channel³⁰. The post and its mirror in the discussion group receive this initial timestamp³⁰.
3. **Post-Event Edit & File Swap:** Once the exam concludes on June 21 and the real

question papers become public, the administrator edits the channel post, replacing the dummy PDF with the real question paper³⁰.

4. **Metadata Mismatch Exploitation:** In the main channel, Telegram appends an "edited" tag to the post³⁰. However, inside the linked discussion group, the mirrored post updates to display the new document **while retaining the original June 18 timestamp and displaying no "edited" label**²⁸.
5. **Extortion and Panic Generation:** Scammers lock the discussion group to prevent comments, pointing users to the un-labeled message as "proof" that they had the leaked exam paper days before the test²⁸. This exploit was used to create panic and extort money from students and parents²⁸.

MeitY Emergency Blocking Order

On June 16, 2026, MeitY issued an emergency blocking order under Section 69A²⁷:

- Directing ISPs to block access to Telegram and its associated domains throughout India²⁷.
- Directing Telegram to immediately disable its message-editing feature for all users within India²⁷.
-

Judicial Determination and Structural Liability

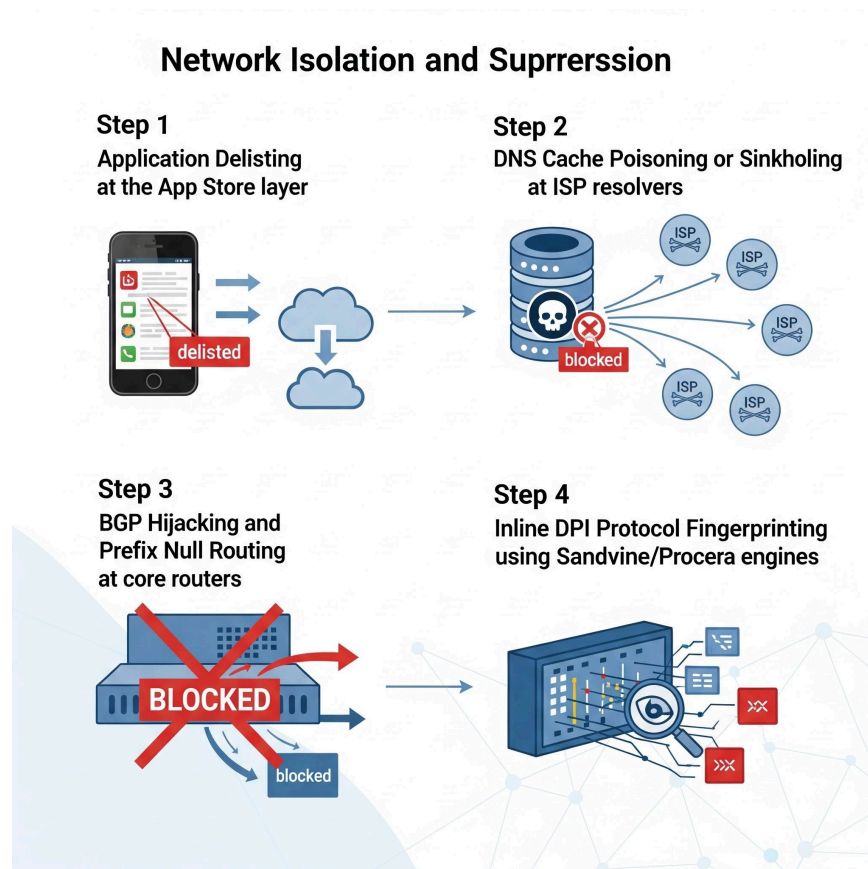
Telegram challenged the blocking order before the Delhi High Court, arguing that Section 69A only allows blocking of specific, identifiable unlawful content, not a platform-wide ban affecting 150 million Indian users⁴¹.

A single-judge bench presided over by **Justice Tejas Karia** dismissed Telegram's petition on June 19, 2026, establishing two key legal principles²⁷:

- **Expansion of the Scope of "Information" under Section 69A:** The Court examined Section 2(1)(v) of the IT Act, which defines "information" to include codes, computer programs, and software²⁷. The Court ruled that because an application or platform functions through software code and communication infrastructure, MeitY is legally empowered under Section 69A to block access to an entire platform if its design prevents narrower enforcement measures²⁷.
- **Proportionality and Architectural Liability:** Applying the proportionality standard from *Anuradha Bhasin (2020)*, the Court held that MeitY's platform-wide blocking order was justified as a measure of last resort²⁷. Because Telegram's architectural features—including high-capacity anonymous channels, automated bot ecosystems, and the timestamp-retention edit flaw—enabled rapid distribution of unlawful content and bypass of targeted takedowns, the platform's design justified the platform-level block during the critical examination period²⁷. The block was subsequently lifted on June 23, 2026, after the examination window closed⁶⁴.

Network Firewalls and App Suppression Mechanisms

When MeitY issues a blocking directive under Section 69A, the Department of Telecommunications (DoT) triggers an automated multi-layer suppression sequence across licensed carrier Internet Service Providers (ISPs), such as Reliance Jio, Bharti Airtel, and Vodafone Idea²⁷.



Step 1: Layer 7 Application Store Geo-fencing and Delisting

The sequence begins with Layer 7 (Application) intervention⁷. The DoT issues directives to Google LLC and Apple Inc. to remove the target application package names (such as `org.telegram.messenger` or `com.whatsapp`) from the Indian Play Store and App Store¹⁷.

Mechanism

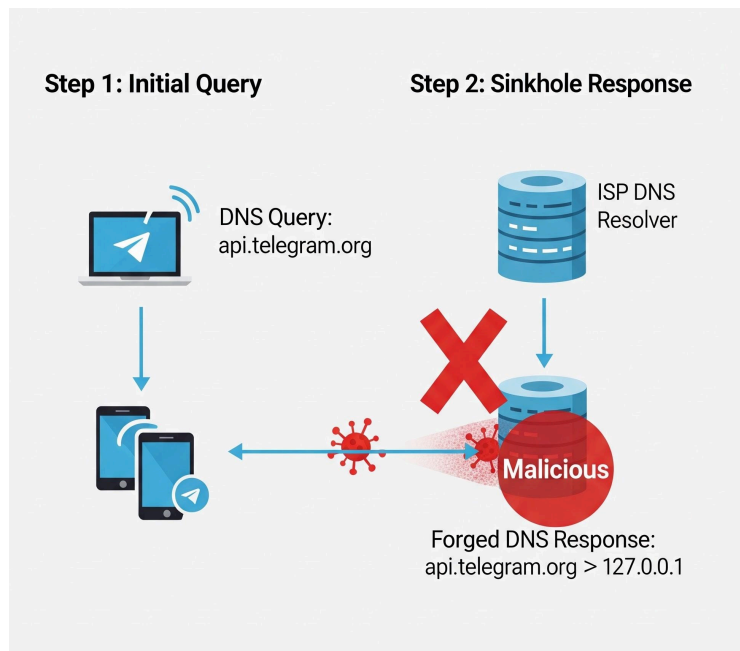
This is enforced at the App Store layer using billing country codes, linked payment profile locations, and real-time client IP locations. Sideloaded older APK versions are common on Android, but iOS device restrictions often prevent new client installations during a ban.

Step 2: Layer 5 DNS Cache Poisoning and Sinkholing

To prevent clients from resolving the domain names of messaging backends, carrier ISPs configure their DNS recursive resolvers to perform DNS Sinkholing⁶⁵.

Mechanism

When an application attempts to resolve domains like `api.telegram.org` or `e.whatsapp.net`, the ISP's recursive DNS resolver intercepts the query⁶⁵. Instead of returning the correct resource record (A or AAAA), the resolver injects a forged IP address⁶⁶.



The messaging client attempts to establish a TCP socket connection with 127.0.0.1 or the sinkhole IP, which fails, preventing communication with the server.

Step 3: Layer 3 BGP Hijacking and IP Access Control Lists (ACLs)

For advanced blocks, carriers deploy Border Gateway Protocol (BGP) routing manipulation and hardware Access Control Lists (ACLs) at transit boundary routers⁶⁷.

BGP Prefix Hijacking / Null Routing (Blackholing)

The ISP's Autonomous System (AS) updates its internal core routing tables by advertising a more specific or shorter prefix path for the IP address blocks assigned to Telegram or WhatsApp⁶⁷. For example, if Telegram operates on subnet 149.154.160.0/20, the ISP's edge routers announce a BGP update for a more specific prefix /24 block (e.g., 149.154.167.0/24) pointing to a Null interface (Null0)⁶⁷. Under standard BGP routing logic, routers prioritize the most specific prefix match, dropping matching traffic at the carrier boundary.

IP Access Control Lists (ACLs)

At the edge routing layer, carriers configure hardware-accelerated IP ACLs to drop inbound and outbound IP traffic matching the known ASN and subnet registers of the target platform. ! Cisco IOS-XE/Juniper Edge Router Config Example ip access-list extended BLACKLIST-TELEGRAM deny ip any 149.154.160.0 0.0.15.255 deny ip any 91.108.4.0 0.0.3.255 permit ip any any

Step 4: Layer 4-7 Deep Packet Inspection (DPI) Protocol Fingerprinting

To counter obfuscation proxies, MTPROTO-zig tunnels, and VPNs, carriers deploy inline Deep Packet Inspection (DPI) systems⁶⁸. These systems perform real-time pattern matching on transit packets⁶⁸.

Protocol Fingerprinting Mechanisms

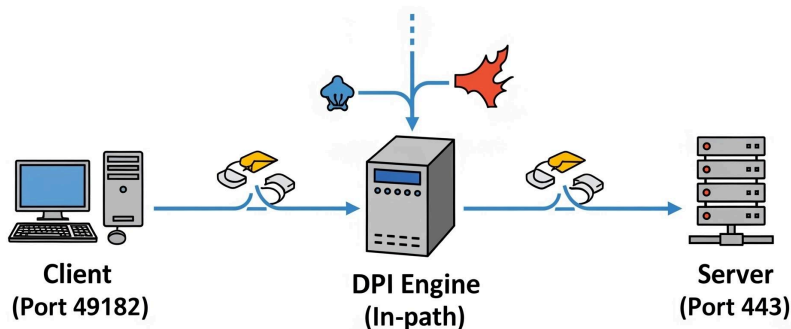
The DPI system parses traffic at the transport and presentation layers to identify signatures unique to specific protocols, such as OpenVPN, WireGuard, and MTPROTO 2.0⁶⁸.

- **TLS SNI (Server Name Indication) Parsing:** During the plain-text TLS handshake (ClientHello), the DPI engine scans the SNI extension field⁶⁹. If the SNI string matches a restricted platform domain, the connection is blocked⁷².
- **Entropy and Handshake Packet Length Verification:** Standard obfuscated protocols often transmit high-entropy payloads lacking clear plaintext identifiers⁶⁸. DPI platforms analyze packet lengths and entropy metrics to identify these connections⁶⁸. For example, WireGuard handshakes use precise packet lengths (148-byte Handshake Initiation and 92-byte Handshake Response), allowing systems to identify and block them⁶⁸.

Injection of TCP RST Packets

Once a target protocol is identified, the DPI system injects spoofed TCP Reset (RST) packets into the connection⁶⁸.

Man-in-the-middle DPI Attack



The DPI engine fabricates two TCP RST packets⁷⁴:

1. One addressed to the Client, mimicking the Server's source IP, sequence number, and port⁷⁴.
2. One addressed to the Server, mimicking the Client's source IP and sequence number⁷⁴.

Both endpoints receive these reset commands, causing them to immediately close the TCP socket session and break the connection⁶⁸.

Layer-Mapped Suppression Sequence

OSI Layer	Target Node / Protocol	Blocking Method	Technical Implementation	Alternative Bypass Path
Layer 7 (Application)	App Store Packages ⁷	Package delisting & geo-fencing ²⁷	Account metadata check, removal of APK/IPA resources ²⁷ .	Sideloaded via F-Droid, PWAs over secure browsers ¹⁷ .
Layer 5 (Session)	DNS Resolvers ⁷	DNS cache poisoning & sinkholing ⁶⁵	Resolving target domains to 127.0.0.1 or loopback ⁶⁶ .	DNS-over-HTTPS (DoH), DNS-over-TLS (DoT).

Layer 3 (Network)	IP Subnet Routing ⁷	BGP hijacking & router-level ACLs ⁶⁷	Advertising /24 null-routes, filtering IP tables ⁶⁷ .	Multi-hop Tor routing, secure VPN tunneling ⁷⁰ .
Layer 4–7 (Cross-Layer)	TCP Ports & Protocols ⁷	DPI fingerprinting & TCP RST injection ⁶⁸	Stateful inspection of packet sizes, injection of spoofed RSTs ⁶⁸ .	TCP desynchronization, packet fragmentation ⁷⁰ .

Off-Grid and Resilient Bypass Topologies

As state-enforced censorship and centralized internet shutdowns grow in scale, system architects and digital-rights advocates have developed decentralized, peer-to-peer (P2P), and off-grid communication topologies that operate independently of centralized ISPs and corporate application stores⁷⁵.

Hardware Mesh Networks: LoRa and Meshtastic Topologies

Long Range (LoRa) technology provides low-power, long-distance wireless communication in license-free Ultra-High Frequency (UHF) bands³⁴. In India, these operate within the **865–867 MHz** spectrum bands⁷⁵.

Physical Layer CSS Modulation

LoRa utilizes Chirp Spread Spectrum (CSS) modulation³⁴. CSS encodes data using frequency chirps that vary continuously over time³⁴. This design provides high resistance to multipath fading, electromagnetic noise, and deliberate signal jamming, allowing low-power transmissions to be received below the thermal noise floor³⁴.

Ad-hoc Mesh Routing

Platforms like **Meshtastic** use LoRa hardware to build ad-hoc, decentralized mesh networks⁷⁵. The routing model utilizes a flooding-gossip protocol⁷⁸. When a LoRa transceiver node receives

a packet, it extracts the payload and inspects the 32-bit unique message identifier⁷⁸. If the packet identifier has not been processed previously (checked against a local Bloom filter to avoid duplicate processing loops), the node decrements the Hop Limit (similar to IPv4 TTL) and rebroadcasts the packet⁷⁸. This allows multi-hop routing across cities without reliance on cellular networks, backhaul fiber, or centralized switches⁷⁵.

Multi-Transport P2P Protocols

Peer-to-peer software architectures run directly on consumer smartphones, utilizing local wireless interfaces to route messages without external servers⁷⁵.

Briar's Multi-Transport Routing Engine

Briar utilizes a multi-transport routing framework that dynamically selects communication channels based on link availability⁷⁵.

- **Online State (Tor Onion Routing):** When internet connectivity is active, Briar routes connections exclusively through the Tor network using cryptographic hidden services (.onion addresses). This ensures that transport-level metadata and user IP locations are masked from ISPs and deep packet inspection³².
- **Offline State (Local Ad-hoc Meshes):** If the internet connection is lost, Briar shifts to its local mesh engine, routing data via **Bluetooth Low Energy (BLE)** and **Wi-Fi Direct**⁷⁵. The application uses a Delay-Tolerant Networking (DTN) store-and-forward architecture. When direct peers are out of range, messages are buffered securely in SQLCipher-encrypted local storage²⁰. When devices move and establish local ad-hoc connections, the encrypted data is synced automatically across peers until it reaches its target⁷⁵.

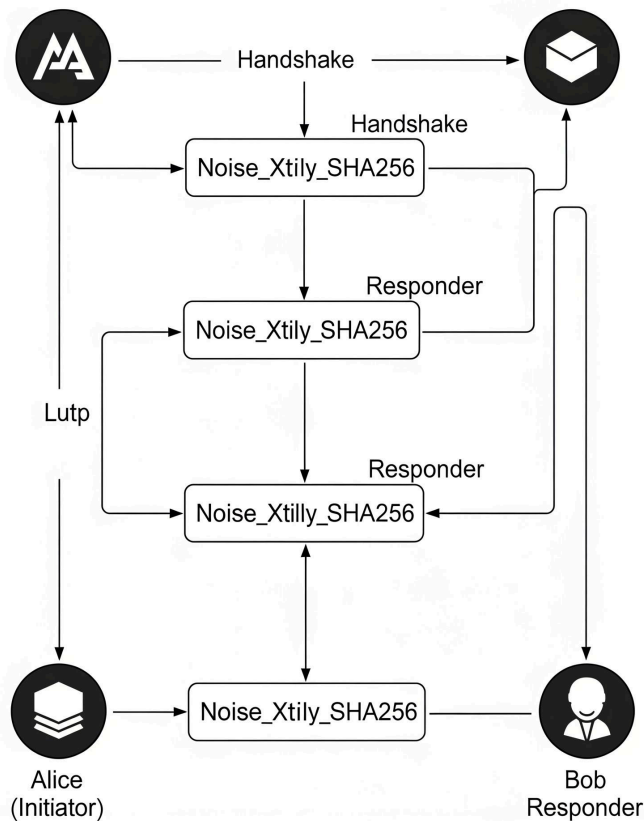
Jack Dorsey's BitChat Protocol

Released in July 2025 as an open-source decentralized messaging system, **BitChat** (styled as bitchat) establishes permissionless local networks without requiring phone numbers, user accounts, or active internet connections⁷⁸.

Cryptographic Layer

BitChat secures all peer-to-peer and mesh communications using the **Noise Protocol Framework**, specifically the Noise_XX_25519_ChaChaPoly_SHA256 pattern⁷⁸. This handshake achieves mutual authentication, payload confidentiality, and forward secrecy⁷⁸.

Noise_XX_25519_ChaChay_Poly_SHA256



Gossip Protocols & Optimized Bloom Filters

BitChat uses a flood-routing protocol optimized for low-bandwidth, high-loss BLE environments⁷⁸. To prevent redundant transmissions and packet storms, each node maintains an **Optimized Bloom Filter** to track active packet signatures⁷⁸. If a received BitchatPacket matches an active filter entry, the packet is silently discarded⁷⁸. Otherwise, the node updates the filter, decrements the packet's 8-bit TTL field, and rebroadcasts the packet⁷⁸.

Internet nostr Gateway Syncing

A key architectural feature of BitChat is its ability to bridge local BLE meshes with global networks⁷⁸. If any node in the BLE mesh gains internet access, the client automatically synchronizes queued local packets with decentralized global **Nostr relays** using NIP-17 direct message formats⁷⁸. This allows localized offline networks to connect with global users once a single gateway node is established⁷⁸.

Alternative Distribution Mechanisms

To bypass geo-fenced application stores, decentralized networks utilize alternative distribution models²⁷.

F-Droid and Cryptographic Sideloads

F-Droid is an independent, community-driven application repository for Android¹⁷. F-Droid client applications communicate directly with repository mirrors hosted on IPFS (InterPlanetary File System) or Onion services. Updates are signed using PG/GPG keys, allowing users to verify application binaries independently of centralized application stores¹⁷.

Progressive Web Applications (PWAs)

PWAs run directly inside standard web browser engines (e.g., WebKit, Blink), escaping standard OS sandbox restrictions and application store review pipelines. PWAs utilize **Service Workers** (background scripts) to execute client-side caching, database persistence via indexed local APIs, and background notification synchronization, enabling full application performance through a standard URL.

Comparison of Resilient Mesh and Bypass Systems

Parameter	Meshtastic (LoRa Mesh)	Briar (P2P Multi-Transport)	BitChat (Local-to-Global P2P)
Primary Transport Layer	Ultra-High Frequency (UHF) Radio ⁷⁵	Bluetooth, Wi-Fi Direct, Tor ⁷⁵	Bluetooth Low Energy (BLE), Wi-Fi Direct ⁷⁸
Physical Infrastructure	Standalone LoRa node hardware ⁷⁵	Consumer smartphone adapters ⁷⁵	Standard Android / iOS mobile chipsets ⁷⁸
Routing Protocol Model	Decentralized flooding-gossip routing ⁷⁸	Ad-hoc DTN store-and-forward ⁷⁵	Gossip routing with Bloom filter limits ⁷⁸
Cryptographic Suite	Symmetric AES-128 / AES-256 ⁷⁵	Signal Double Ratchet over Tor ³	Noise XX Handshake over Curve25519 ⁷⁸

Dependency Context	Zero internet dependency ⁷⁵	Hybrid online/offline switching ⁷⁵	Integrated Nostr bridging when online ⁷⁸
Anonymity Profile	High anonymity (no phone numbers or IDs) ⁷⁵	Strong onion-routing metadata isolation ⁷⁵	Pseudonymous keys (no registration) ⁷⁸

Conclusions and Strategic Recommendations

This comparative investigation highlights the ongoing tension between cryptographic privacy, state security demands, and network censorship. The shift of Meta's Instagram DMs away from E2EE in May 2026 demonstrates how platforms are forced to modify their security architectures to meet strict regulatory takedown timelines, such as the US Take It Down Act⁵. Concurrently, the June 2026 Delhi High Court ruling in *Telegram FZ LLC v. Union of India* has set a significant legal precedent in Indian Cyber Law²⁷. By interpreting the term "information" under Section 2(1)(v) of the IT Act, 2000 to include entire software platforms, the judiciary has expanded the scope of Section 69A²⁷. The ruling introduces **architectural liability**, meaning a platform's design choices—such as its message-editing features or bot ecosystems—can justify platform-wide blocking orders if they are found to hinder content-specific moderation during periods of high public risk²⁷.

To navigate this landscape, systems architects, cybersecurity researchers, and legal policy experts should consider the following recommendations:

Design and Architectural Principles for Messaging Platforms

- **Implement Immutable Audit Logs for Message Edits:** To prevent "after-the-event" evidence fabrication scams (like those seen in the NEET-UG Telegram exploit), messaging platforms must avoid un-labeled updates³⁰. Every message edit should update the cryptographic envelope of the message and append a visible "edited" flag across all linked groups, sub-chats, and synced databases, ensuring consistency between client-side displays and database states³⁰.
- **Deploy Zero-Trust Notification APIs:** To prevent local metadata and plaintext leaks via system logging APIs, applications must sanitize payload handoffs to OS notification services³⁵. High-sensitivity messages should only transmit generic notification indicators (e.g., "New Message Received") to the OS³⁵. The actual message content should only be decrypted and displayed when the user unlocks the device and opens the application within its secure memory sandbox².

- **Evaluate Hybrid Decentralized and Peer-to-Peer Gateways:** As centralized platforms remain vulnerable to platform-wide blocking under Section 69A, systems architects should investigate hybrid, multi-transport frameworks²⁷. By integrating fallback channels—such as local BLE meshes, Wi-Fi Direct, and decentralized relays (e.g., Nostr, Tor)—communications can continue even during localized network shutdowns or platform-level bans⁷⁵.

Balancing Legal Compliance with Cryptographic Integrity

- **Mitigate Compliance Risks under Rule 4(2):** Developers of E2EE platforms should work to establish technical compliance frameworks that protect user privacy⁴⁸. If ordered to trace the first originator of a message, platforms should look to use privacy-preserving cryptographic techniques (such as zero-knowledge proofs and blind signatures) to verify source properties without compromising the deniability or forward secrecy of the underlying protocol².
- **Establish Transparent Legal Response Protocols:** Intermediaries must operate dedicated, trained Law Enforcement Response Teams (LERT) to verify that all government data requests comply with the statutory grounds of Section 69A and the IT Act¹. Any data production must be narrowly tailored, producing only the metadata required by law while protecting the cryptographic integrity of transit payloads¹.

By adopting these principles, systems architects and legal experts can build secure, resilient communication technologies that defend user privacy while navigating the regulatory demands of sovereign states.